

## **ASSIGNMENT 9 — PyTest Integration with HTML Reporting**

### **1. Experiment Title**

#### **PyTest Integration with HTML Reporting and Failure Screenshots**

### **2. Problem Statement**

Convert the Selenium automation setup to run using **PyTest**. Implement PyTest fixtures for browser initialization and teardown, execute the test suite from the terminal, and generate an HTML execution report containing screenshots when a test fails.

### **3. Objective**

- To integrate Selenium automation with PyTest.
- To use PyTest fixtures for browser setup and teardown.
- To execute tests from the terminal.
- To generate an HTML execution report.
- To capture screenshots when tests fail.

### **4. Tools, Software and Concepts Used**

#### **Tools & Software:**

- Python
- Selenium WebDriver
- PyTest
- pytest-html
- Google Chrome
- VS Code / PyCharm

#### **Concepts:**

- PyTest
- Fixtures
- Setup and Teardown

- Selenium WebDriver
- HTML Reporting
- Screenshots
- Test Execution
- Assertions

## 5. Source Code / Implementation Steps

### Project Structure

Assignment\_9\_PyTest\_Reporting

```
|
|— conftest.py
|— test_assignment9.py
└— reports
```

#### **conftest.py**

```
import pytest

from selenium import webdriver

@pytest.fixture
def driver():

    driver = webdriver.Chrome()

    driver.maximize_window()

    yield driver

    driver.quit()
```

#### **test\_assignment9.py**

```
from selenium.webdriver.common.by import By

def test_successful_login(driver):

    driver.get("https://www.saucedemo.com/")

    driver.find_element(
```

```
        By.ID, "user-name"
    ).send_keys("standard_user")
    driver.find_element(
        By.NAME, "password"
    ).send_keys("secret_sauce")
    driver.find_element(
        By.XPATH, "//input[@type='submit']"
    ).click()
    assert "/inventory.html" in driver.current_url
```

### **Install HTML Reporting Plugin**

```
pip install pytest-html
```

### **Run the Test**

```
pytest -v --html=reports/report.html --self-contained-html
```

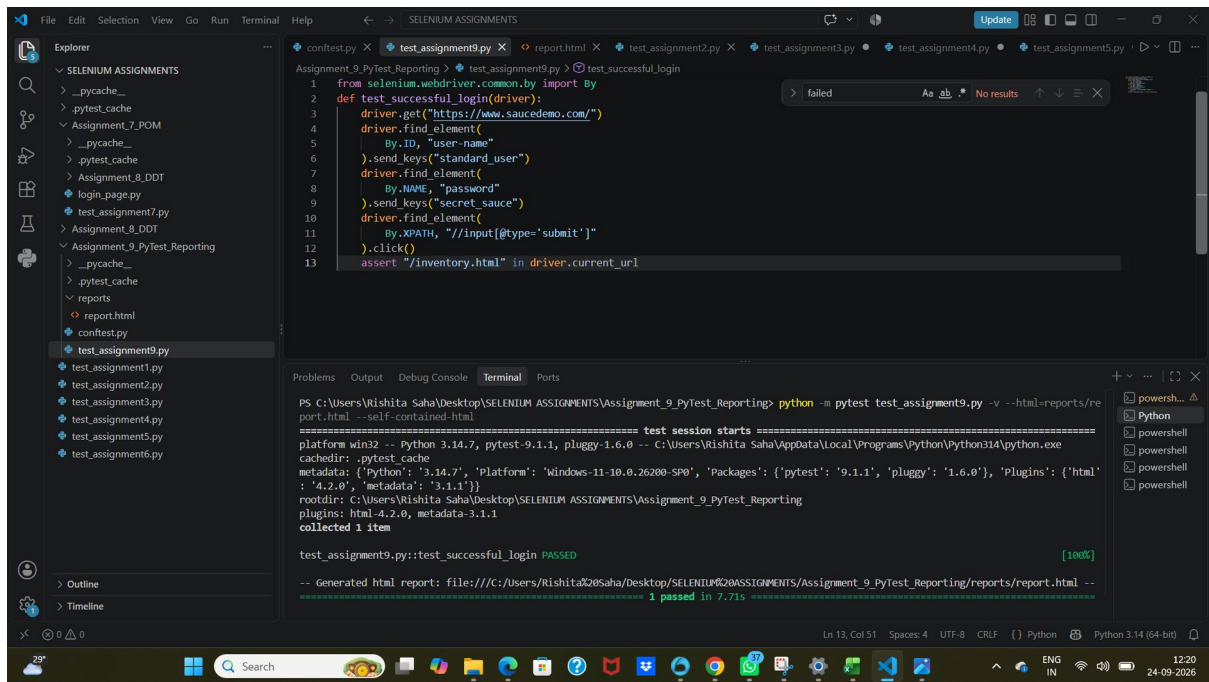
The report will be generated inside:

Assignment\_9\_PyTest\_Reporting

└─ reports

└─ report.html

## **6. Output with Clearly Labelled Screenshots**



## Successful PyTest Execution

Show the terminal after running:

## 7. Result

The Selenium test suite was successfully integrated with **PyTest**. A PyTest fixture was used to initialize and close the browser, and an HTML execution report was generated using the pytest-html plugin.

## 8. Observation

- Fixtures provide reusable setup and teardown functionality.
- The browser is initialized before the test and closed after execution.
- PyTest allows the Selenium tests to be executed directly from the terminal.
- pytest-html generates a detailed HTML execution report.
- Screenshots can be incorporated into reports for failed test cases when failure-capture logic is implemented.

## 9. Conclusion

This experiment demonstrated the integration of **Selenium with PyTest and HTML reporting**. PyTest fixtures simplified browser management, while HTML reporting provided a structured way to review test execution results.